

Lecture 1: Logistic Regression & Linear Classification

Alexis BOGROFF

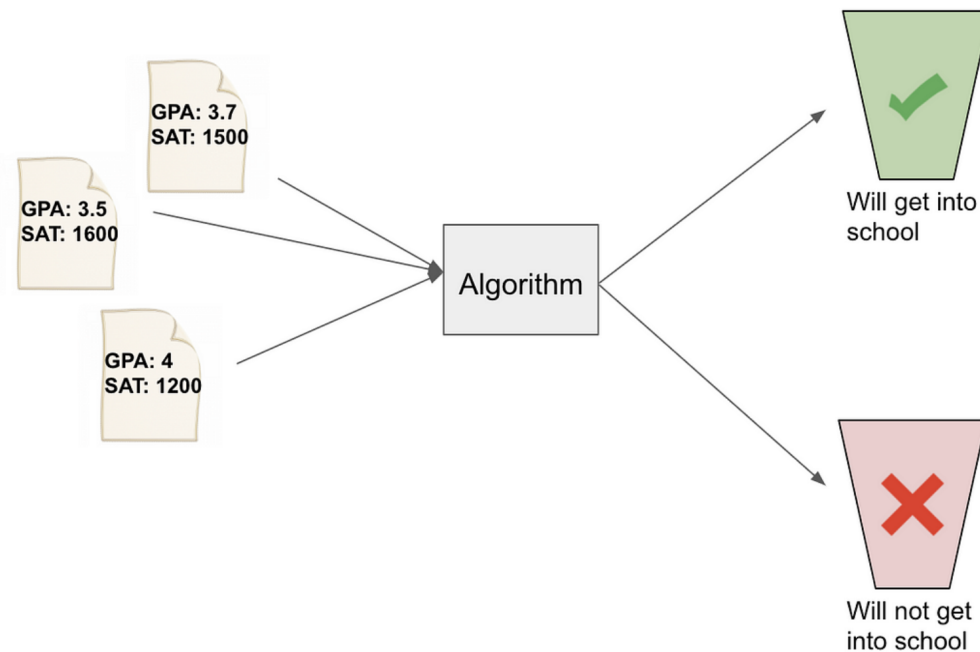
Course on Classification Problems @Albert School

January 20, 2026

- **Define the Problem:** Clearly specify the objective and collect quality, representative data.
- **Data Splitting:** Divide the data into training and test sets (prevent data leakage).
- **Data Preparation:** Preprocess data through cleaning, normalization, encoding, and feature engineering.
- **Model Training:** Use the correct algorithms to learn patterns from training data.
- **Model Evaluation:** Validate the model using metrics (Accuracy, Log-Loss).
- **Deployment and Monitoring:** Deploy the model into production.

Introduction to Classification Problems

- Classification is a supervised learning task where the output variable is **categorical**.
- Examples:
 - Spam detection (Binary: Yes/No).
 - Churn prediction (Binary: Churn/Stay).
 - Handwritten digit recognition (Multi-class: 0-9).



What does Classification Data look like?

The Structure of a Supervised Dataset

- **Features (X):** The input variables (columns) describing the observation.
- **Target (Y):** The label we want to predict. In classification, Y is discrete.

Example: Titanic Dataset (Binary Classification)

Age (X_1)	Fare (X_2)	Class (X_3)	Survived? (Y)
22	7.25	3	No (0)
38	71.28	1	Yes (1)
26	7.92	3	Yes (1)
35	53.10	1	Yes (1)
⋮	⋮	⋮	⋮

Goal: Learn a function f such that $f(\text{Age}, \text{Fare}, \text{Class}) \approx \text{Survived}$.

A misnomer: Despite its name, it is a **Classification** algorithm, not a Regression one.

- **Role:** It predicts the probability that an observation belongs to a specific category (Class 0 or Class 1).
- **Type:** It is a **Linear Model**. It assumes a linear relationship between input features and the log-odds of the outcome.
- **Status in AI:** It is considered the "Hello World" of classification. Simple, fast, but surprisingly powerful.

"If you can't beat Logistic Regression with your fancy Deep Learning model, use Logistic Regression."

Where is Logistic Regression actually used in the industry?

1. Banking & Finance

- **Credit Scoring:** Will this customer default on their loan?
- **Why LogReg?** Regulations require **explainability**. You must explain why a loan was refused.

2. Medicine

- **Diagnosis:** Based on symptoms, does the patient have Disease X?
- **Risk Factors:** Calculating Odds Ratios (e.g., "Smoking increases risk by 5x").

3. Marketing

- **Churn Prediction:** Will this subscriber cancel their subscription next month?
- **Ad Click:** Will the user click on this banner? (CTR).

Strategic Choice: Why use Logistic Regression?

Why choose a simple linear model over complex ones (Random Forest, Neural Networks)?

Feature	Logistic Regression	Complex Models (e.g. XGBoost)
Interpretability	High (White Box)	Low (Black Box)
Training Speed	Very Fast	Slower
Data Needed	Small datasets ok	Needs massive data
Performance	Good on simple data	Best on complex data

Decision Rule: Start with Logistic Regression as a **Baseline**. Only switch to complex models if the performance gain is significant and worth the loss of interpretability.

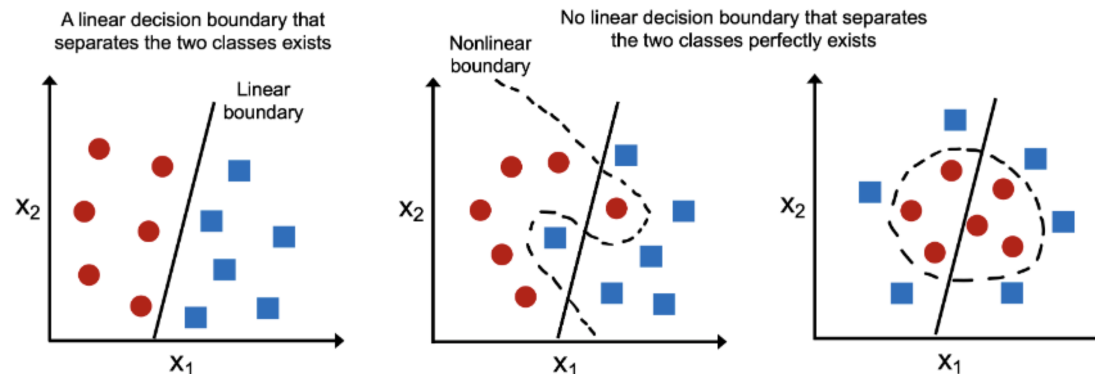
Logistic Regression draws a **straight line** (or plane) to separate classes.

When it works:

- Classes are "linearly separable".
- Example: Separating huge tumors (Malignant) from small ones (Benign).

When it fails (The XOR Problem):

- If classes are arranged in a circle or a checkerboard pattern.
- A single straight line cannot separate them.
- **Solution:** Feature Engineering (add x^2) or Non-linear models (Trees, Neural Nets).



The Challenge: How can we predict a binary probability P using a linear combination of features?

Hypothesis (Attempt #1): Let's try to equate P directly to the linear equation.

$$\text{Let's test: } P(Y = 1|X) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$$

Why this is a "Dead End":

1. Mathematical Impossibility (Range Mismatch):

- The linear term $\beta^T X$ goes from $-\infty$ to $+\infty$.
- A probability must stay in $[0, 1]$.
- **Result:** For extreme X , the model predicts $P = -0.5$ or $P = 1.2$. **It breaks.**

2. Behavioral mismatch:

- A straight line assumes constant impact. In reality, probabilities "saturate" (diminishing returns).

Conclusion: We cannot model P directly. We need to transform it into an unbounded value first.

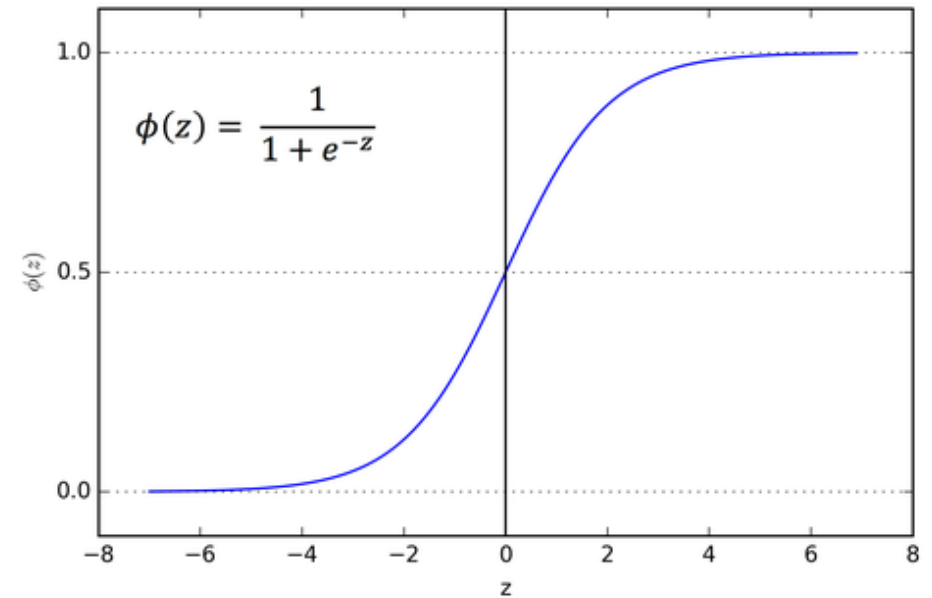
The Logistic Model: Sigmoid Function

- We model the probability $P(Y = 1|X)$ using the **sigmoid** function $\sigma(z)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Where z is the linear combination of features (the logit):

$$z = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$$

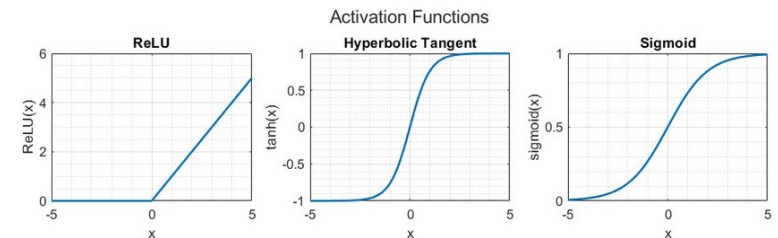


Key property: $\sigma(0) = 0.5$

Why Sigmoid? Comparison with other functions

Why don't we use functions popular in Deep Learning like **ReLU** or **Tanh**?

- **Sigmoid:** Ranges in $[0, 1]$.
- **Tanh (Hyperbolic Tangent):** Ranges in $[-1, 1]$.
 - **Problem:** A probability cannot be negative ($P < 0$ makes no sense).
- **ReLU (Rectified Linear Unit):** Ranges in $[0, +\infty)$.
 - Formula: $f(x) = \max(0, x)$.
 - **Problem:** Probabilities cannot exceed 1 ($P > 1$ makes no sense).



Only Sigmoid maps strictly to probability space.

Beyond the range $[0, 1]$, Sigmoid is the mathematically consistent choice for Linear Classification.

- **1. Smoothness (Differentiability):** Unlike the "Step Function" (Perceptron) which jumps from 0 to 1, Sigmoid is smooth. This allows us to compute **gradients** for optimization.
- **2. The Log-Odds assumption:** If we assume that the *Log-Odds* are linear:

$$\log \left(\frac{p}{1-p} \right) = \beta^T X$$

Then, by inverting this equation, we **mathematically obtain** the Sigmoid function:

$$p = \frac{1}{1 + e^{-\beta^T X}}$$

Conclusion: Sigmoid is not arbitrary; it is the natural inverse of the Linear Log-Odds.

To understand the coefficients β , we need to look at the **Odds**:

- **Probability:** $P \in [0, 1]$
- **Odds:** $\frac{P}{1-P} \in [0, +\infty)$ (How much more likely is success than failure?)
- **Log-Odds (Logit):** $\log\left(\frac{P}{1-P}\right) \in (-\infty, +\infty)$

The Linear Link:

$$\log\left(\frac{P(Y=1)}{P(Y=0)}\right) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$$

Logistic Regression is a Linear Regression on the Log-Odds.

Deriving the Sigmoid Function (1/2)

How do we get from the **Log-Odds** back to the **Probability** P ?

1. Start with the **Linear Assumption**:

$$\log\left(\frac{P}{1-P}\right) = \underbrace{\beta_0 + \beta_1 X_1 + \dots}_{\text{Let's call this } z} \Rightarrow \log\left(\frac{P}{1-P}\right) = z$$

2. Remove the Logarithm (**Exponentiate both sides**):

$$\frac{P}{1-P} = e^z$$

3. Isolate P (**Cross-multiply**):

$$P = e^z(1 - P)$$

$$P = e^z - P \cdot e^z$$

4. Group terms with P on the left:

$$P + P \cdot e^z = e^z$$

$$P(1 + e^z) = e^z$$

5. Solve for P:

$$P = \frac{e^z}{1 + e^z}$$

6. **Final Polish (Divide by e^z):** To get the standard "Sigmoid" form, we multiply top and bottom by e^{-z} :

$$P = \frac{1}{e^{-z} + 1} = \sigma(z)$$

Conclusion: The Sigmoid is the inverse of the Logit.

- The decision boundary is the set of points where the model is uncertain ($P = 0.5$).
- Mathematically, $\sigma(z) = 0.5 \iff z = 0$.

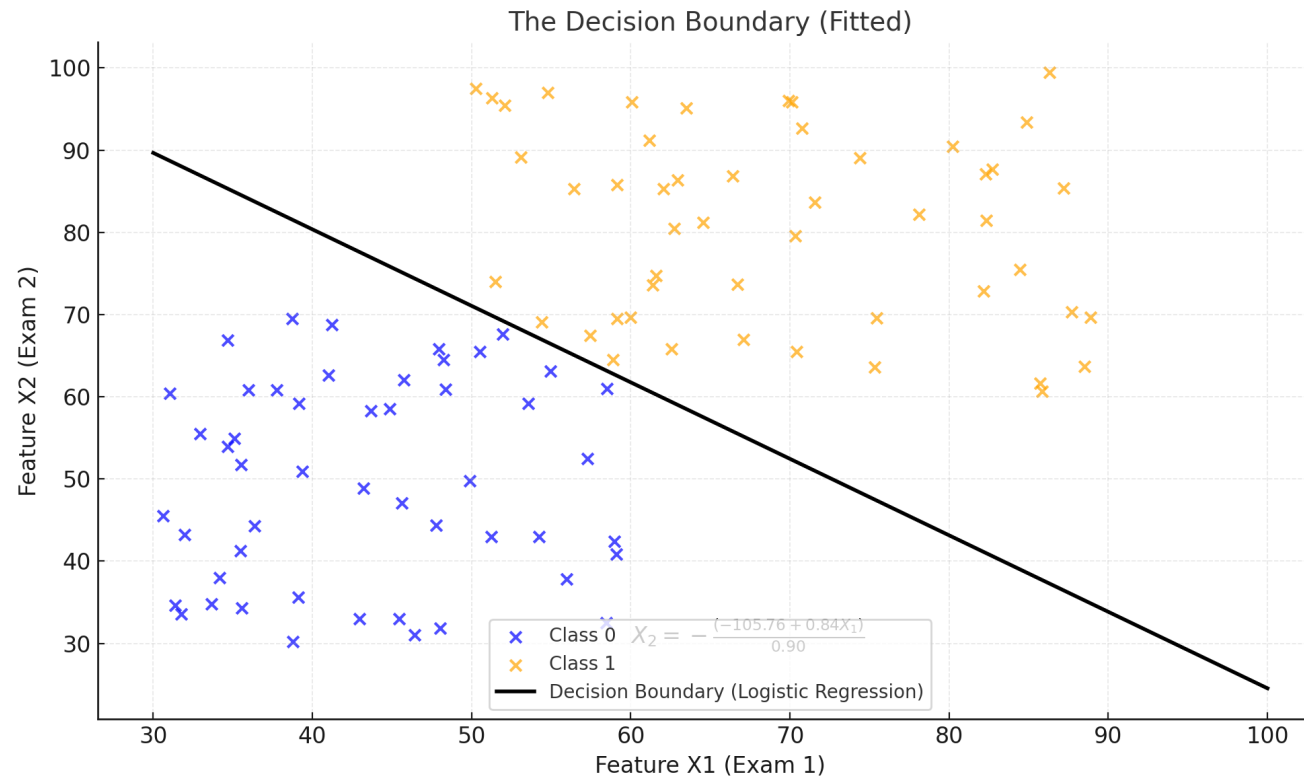
$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0$$

- This equation represents a **Line** (in 2D) or a **Hyperplane** (in higher dimensions).

Prediction Rule:

- If $\beta^T X > 0 \Rightarrow P > 0.5 \Rightarrow \hat{Y} = 1$
- If $\beta^T X < 0 \Rightarrow P < 0.5 \Rightarrow \hat{Y} = 0$

Visualizing the Decision Boundary



- Points **on** the line have 50% probability.
- Points **far** from the line have probabilities close to 0 or 1.

The superpower of Logistic Regression is **Explainability**. How do we interpret a weight β_j ?

Math Reminder:

$$\log(\text{Odds}) = \beta_0 + \beta_1 X_1 + \dots$$

If we increase X_1 by 1 unit:

- The **Log-Odds** increase by β_1 .
- The **Odds** are multiplied by e^{β_1} (this is called the **Odds Ratio**).

Imagine we fit a model to predict survival:

$$\text{Model: } \beta_{\text{gender_male}} = -2.5, \quad \beta_{\text{age}} = -0.04$$

1. Interpreting Gender ($\beta = -2.5$):

- Odds Ratio = $e^{-2.5} \approx 0.08$.
- *"Being male multiplies the odds of survival by 0.08."*
- (In other words: Men are $\approx 12\times$ less likely to survive than women, all else equal).

2. Interpreting Age ($\beta = -0.04$):

- Odds Ratio = $e^{-0.04} \approx 0.96$.
- *"For every extra year of age, the odds of survival decrease by 4%."*

Loss: Can we use Mean Squared Error (MSE)?

The Question: Since $\hat{y} \in [0, 1]$ and $y \in \{0, 1\}$, why not use the standard regression error?

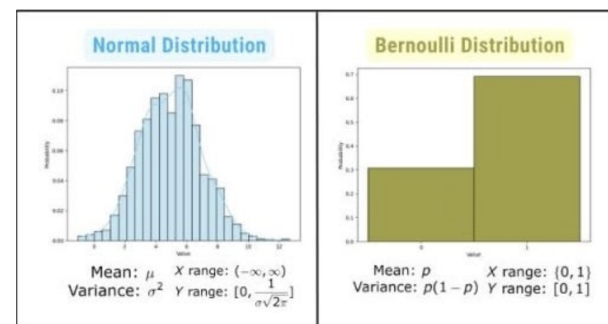
$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

1. It is technically possible:

- The function is differentiable.
- Gradient descent can run on it.

2. But it is statistically incorrect:

- MSE assumes data follows a *Gaussian* (Normal) distribution.
- Binary classification instead follows a **Bernoulli** distribution ($y \sim \text{Bernoulli}(p)$).
- Maximizing Bernoulli likelihood naturally leads to Cross-Entropy, not MSE.



Gaussian (MSE) vs Bernoulli (LogLoss)

Problem 1: The Vanishing Gradient (Saturation)

Why does MSE learn slowly? Let's look at the gradients (derivative of Loss w.r.t weights w).

With Cross-Entropy (Log-Loss):

$$\nabla_w \mathcal{L}_{CE} = (\hat{y} - y) \cdot x$$

If prediction is wrong ($\hat{y} \approx 1, y = 0$), the gradient is large (≈ 1). The model learns fast.

With MSE:

$$\nabla_w \text{MSE} = (\hat{y} - y) \cdot \underbrace{\sigma'(z)}_{\text{Problem!}} \cdot x$$

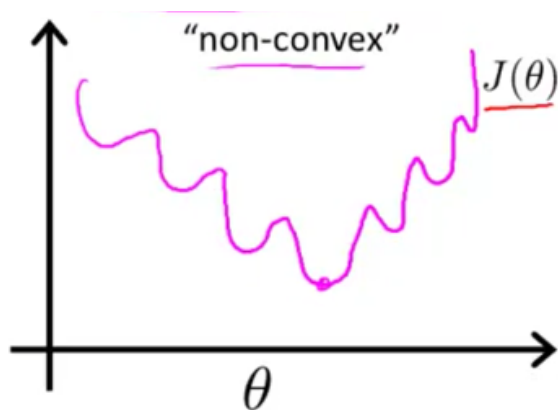
Since $\sigma'(z) = \hat{y}(1 - \hat{y})$, the gradient goes to **zero** if the prediction is confident (close to 0 or 1), even if it is wrong.

Consequence: If the model is "confidently wrong" using MSE, the gradient vanishes, and the model stops learning.

Problem 2: Non-Convexity (Local Minima)

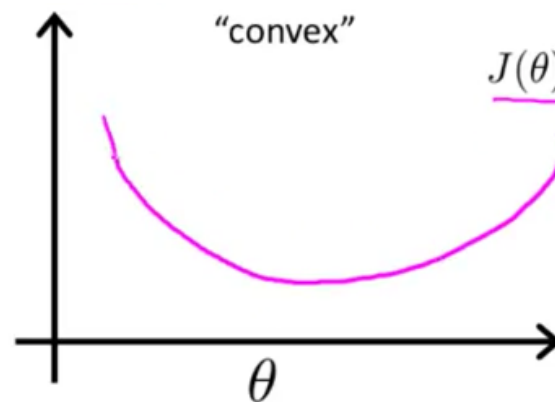
MSE + Sigmoid = Non-Convex

- The combination creates a "wavy" error surface.
- Gradient descent can get stuck in **local minima** (valleys that are not the lowest point).



Cross-Entropy + Sigmoid = Convex

- The surface is shaped like a bowl.
- Guaranteed to find the global optimum (minimum error).



- **Likelihood function (MLE):** We want to maximize the probability of the observed data.

$$L(\beta) = \prod_{i=1}^n P(y_i | x_i, \beta) = \prod_{i=1}^n \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}$$

- **Log-Likelihood:** Easier to differentiate (turns products into sums).

$$\ell(\beta) = \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

- **Cost Function (Binary Cross-Entropy):**
 - Maximizing Likelihood $\Leftrightarrow$ Minimizing Negative Log-Likelihood.
 - This is known as the **Log-Loss**:

$$J(\beta) = -\frac{1}{n} \ell(\beta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

The Bernoulli Distribution: The "Compact" Form

The piecewise definition is clear but hard to manipulate mathematically:

$$P(X = k) = \begin{cases} p & \text{if } k = 1 \\ 1 - p & \text{if } k = 0 \end{cases}$$

The "Classic" One-Line Formula: To make calculus easier (especially for Likelihood), we use this compact form:

$$\mathbb{P}(X = k) = p^k(1 - p)^{1-k}$$

Verification (Why it works):

- **Case** $k = 1$: $p^1(1 - p)^{1-1} = p \cdot 1 = p$ ✓
- **Case** $k = 0$: $p^0(1 - p)^{1-0} = 1 \cdot (1 - p) = 1 - p$ ✓

This formula unifies both cases, making it perfect for derivatives!

There is no closed-form solution (like in Linear Regression). We must use iterative optimization.

- **Update Rule:** $\beta_{new} = \beta_{old} - \alpha \nabla J(\beta)$
- **Gradient:** It turns out the gradient looks very similar to linear regression!

$$\frac{\partial J(\beta)}{\partial \beta_j} = \sum_{i=1}^n (\hat{y}_i - y_i) x_{ij}$$

- **Intuition:**
 - $(\hat{y}_i - y_i)$ is the prediction error.
 - We adjust weights proportional to the error and the feature value.

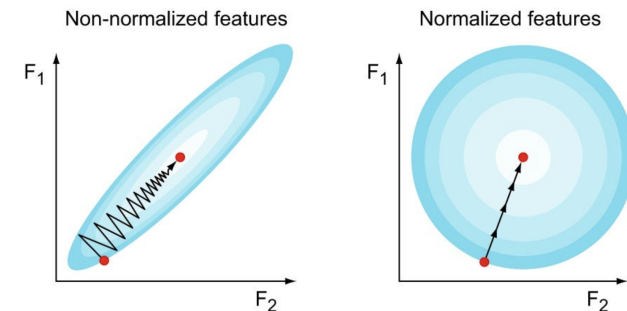
Warning: Logistic Regression uses Gradient Descent optimization.

The Problem:

- Imagine Feature A = "Salary" (Range: 20k - 100k).
- Imagine Feature B = "Age" (Range: 18 - 80).
- The gradients for Salary will be huge compared to Age.
- **Result:** The model will struggle to converge or will be biased towards larger numbers.

The Solution: Always apply **Standardization** (StandardScaler) before training:

$$X_{scaled} = \frac{X - \mu}{\sigma}$$



Unscaled data = Elongated valley (slow). Scaled data = Round bowl (fast).

What if we have more than 2 classes? (e.g., Digit Recognition 0-9, Iris Species)

Binary Classification

- $Y \in \{0, 1\}$
- One decision boundary.
- Uses **Sigmoid**.

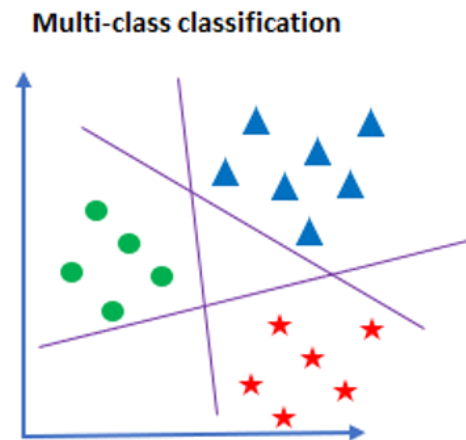
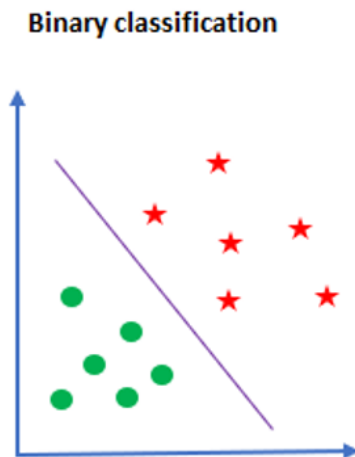
Multi-Class Classification

- $Y \in \{0, 1, \dots, K - 1\}$
- Multiple boundaries dividing the space.
- Two main strategies:
 1. One-vs-Rest (OvR)
 2. Multinomial (Softmax)

Strategy 1: One-vs-Rest (OvR)

Also called *One-vs-All*.

- **Idea:** Decompose the problem into K binary problems.
- For class k , we train a binary classifier: "Is it class k or not?" (vs all others).
- **Prediction:** Run all K models and pick the class with the highest probability score.



Advantage: Simple to implement with any binary classifier (Logistic, SVM, etc.).

Instead of training multiple models, we generalize Logistic Regression directly to output a probability distribution.

- **Softmax Function:** Generalization of Sigmoid for K classes.

$$P(Y = k|X) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

- **Properties:**

- All probabilities are positive and sum to 1 ($\sum P_k = 1$).
- We minimize the **Categorical Cross-Entropy** loss.

Note: In `sklearn`, `LogisticRegression` uses OvR by default but can switch to `multi_class='multinomial'`.

Objective: Go deep into `sklearn` to interpret and visualize the model.

1. Data Setup:

- Load the `iris` dataset (keep only 2 classes and 2 features for 2D viz).
- Split data into Train/Test.

2. Model & Coefficients:

- Fit a `LogisticRegression`.
- Retrieve `.coef_` (β) and `.intercept_` (β_0).
- Write the equation of the boundary $z = 0$.

3. Advanced Visualization (The Goal):

- Scatter plot your training data.
- **Decision Boundary:** Plot the straight line separating the classes.
- **Probability Contours:** Use `predict_proba` on a meshgrid to display the probability gradient (colormap) in the background.

4. Analysis:

- Predict a new point. Compare `predict()` vs `predict_proba()`.